Heesch numbers and maximal coronas of small polyforms with certificates and a Lean-verified SAT encoding

Tatsuhiko Takimoto
Independent researcher

Version 2.1, 23 October 2026

Abstract

For a fixed selection of 3,964 shapes from Kaplan's tables of unmarked polyforms, we give 6,574 corona witnesses and 5,019 upper-bound proofs accepted by drat-trim. A Lean 4 soundness theorem for the SAT encoding covers the square, hexagonal and triangular grids; six theorem statements are accepted in six kernel runs using five implementations. A finite comparison connects all 5,019 checked CNFs to the Lean definition, and a separately written encoding agrees on every value. The proof checking and file comparison remain external to Lean. We also count maximal coronas for 22 selected shapes, distinguishing H_{h} (holes permitted in the outermost layer) from H_{c} (no holes). Two implementations agree completely on 28 of the 30 shape–variant combinations; the other two totals are available from one implementation only. In the fully compared cases every fixed extendable inner part has more than one outer completion, while exactly six H_{h} cases and four H_{c} cases have a unique inner part. Inner enumeration has checked DRAT exhaustion proofs in those 28 cases; the outer counts rely on agreement of separate counting implementations sharing the same encoding and geometric specification. A further, restricted search of 3,442 polyominoes with 20–28 cells yields no example with finite $H_{\mathrm{h}} = 4$.

1 Introduction

A shape that does not tile the plane may still be surrounded by copies of itself. Its Heesch number is the largest number of complete layers (coronas) of congruent copies that can be placed around it. The problem of which values occur goes back to Heesch [8]; Mann's survey [15] describes its history and gives edge-marked hexapillars with Heesch numbers 4 and 5, Fontaine [7] found an infinite family of figures with Heesch number 2, and Bašić [1] constructed a figure with Heesch number 6. Kaplan's recent account [11] describes the present state of the problem in the plane; related questions have been studied in the hyperbolic plane [14] and in three dimensions [2].

Computation has been central to this work. Mann and Thomas computed Heesch numbers of edge-marked polyforms [16]. Kaplan [10] encoded the existence of all coronas up to a given order as a single SAT problem, distinguished the hole-free Heesch number H_{c} from the number H_{h} in which the outermost corona may contain holes, and computed both values for unmarked polyominoes, polyhexes and polyiamonds, publishing the tables, the shapes and the software [12]. The question studied here, the definitions of coronas and of $H_{\mathrm{c}}, H_{\mathrm{h}}$, the idea of a SAT search over all layers at once, and every reference value we compare with are taken from Kaplan's work. The Heesch challenge of Layr-Labs/Eigen [13] already attaches checkable certificates to Heesch numbers: it checks corona witnesses geometrically, regenerates CNFs from a shape and a layer count and compares their digests, converts DRAT proofs to LRAT and checks them with the verified checker cake_lpr [20], and records verified upper-bound proofs for individual shapes with high Heesch numbers. At the commit we consulted, its documentation states that an external review of the correspondence between the geometry and the CNF is still pending. Its census of 3,943 known non-tilers is the same size as the complete, small-size portion of our selection; the additional selected shapes have high Heesch numbers.

A DRAT proof certifies that one particular CNF formula is unsatisfiable. Whether that formula expresses "there is no K-layer corona" is a separate question, which is usually answered by the correctness of the program that generated it. This article addresses that question for one encoding, on all three grids,

with a theorem proved in Lean, and connects the theorem to the files that were actually checked. Its contributions are:

1. for a fixed, hash-identified selection of 3,964 shapes from Kaplan's tables, a lower-bound witness and an upper-bound certificate for every value of H_c and H_h, with all 5,019 proofs accepted by drat-trim [21] (Sections 6 and 8);
2. a Lean 4 statement and proof that the encoding is sound on the square, hexagonal and triangular grids, for every order of the at-most-one counters and every admissible set of hole-blocking clauses (Sections 4 and 5);
3. a finite comparison showing that each of the 5,019 checked CNF files is, up to renaming and reordering, the Lean-defined CNF (Section 7);
4. counts of maximal coronas and of their extendable prefixes for the 22 shapes attaining the largest H_h in each selected grid range, with separate implementations agreeing completely in 28 of 30 shape–variant combinations (Section 9);
5. a documented search of a fixed set of 3,442 polyominoes with 20–28 cells, with no finite-$H_h = 4$ example obtained (Section 10).

Formal verification of SAT encodings and of reductions from geometry to SAT has clear precedents: Codel, Avigad and Heule verified encodings, including the sequential counter, in Lean [4, 5]; Holliday, Norman, Pacuit and Zahedian proved in Lean that the existence of a voting rule yields a model of their CNF [9]; Subercaseaux, Nawrocki, Gallicchio, Codel, Carneiro and Heule verified the reduction behind the empty hexagon number and connected it to a cake_lpr check [18]. Our formalization applies this approach to the coronas, holes and candidate placements of the Heesch problem on three grids. Section 11 compares the guarantees in more detail.

2 What is claimed, and on what basis

Table 1 lists the claims and the kind of evidence behind each. Who ran which check, when, and with which tool versions is recorded in the verification record that accompanies this article (`VERIFICATION-RECORD.md`). The article has not undergone human expert review.

Table 1: Claims and their basis. Finite checks use programs that are not formally verified.

Claim	Basis	Range
Lean proof		
CNF soundness (Theorem 4)	Six theorems; standard axioms `propext`, `Classical.choice`, `Quot.sound`; six kernel runs (five implementations)	Three grids, all $K \geq 1$, counter orders and valid blocks
Square-grid instance equals the earlier square-only definition	Lean proof outside the Comparator statement, same axioms	All arguments
Prose		
Truncation and $H_h - 1 \leq H_c \leq H_h$; counting conventions and minimality (Proposition 6)	Lemma 2; the arguments in Section 9	Stated domains
Disc criteria used in counting: hole tests suffice for inner layers; Euler-characteristic tests for the outer H_c layer	Elementary topology (Section 9.2)	Hexagonal and triangular outer layers; all inner layers
Empty-block UNSAT excludes every (P0)–(P3) patch, without disc conditions	Consequence of the proof outline of Theorem 4, stronger than its Lean statement	Empty block list

Table 1 continued

Claim	Basis	Range
AI review		
Lean definitions express the geometry and encoding faithfully	Independent reading against the specification and Kaplan; "faithful with caveats"	Statement file
Finite computation		
Upper bounds and file correspondence	drat-trim accepts 5,019 proofs; Python transcriptions match 64 Lean outputs, then all 5,019 CNFs up to renaming and reordering	Table 5
Lower bounds and independently encoded values	Two geometric checkers accept 6,574 witnesses; second encoding agrees on all values, without proof logs	3,964 shapes
Maximal-corona prefix enumeration	Coordinate-wise agreement; checked DRAT exhaustion of the enumeration CNFs on both sides (these CNFs are outside the Lean theorem and the file comparison); every solution checked geometrically	28 combinations; #315 prefixes agree without checked DRAT
Outer completions and total counts	Two separate counting implementations agree per inner part; no proof certificate for the outer counts	28 totals agree; two #315 totals from one side
Restricted search	SAT screening and explicit periodic tilings; 14 solver cross-checks and two sampled DRAT checks	3,442 shapes; Table 7
Not claimed	Exhaustiveness beyond the listed inputs; off-grid placements; Lean-internal UNSAT or file-comparison proofs; formally verified counting	

The end-to-end statement for the tabulated values rests on: (1) the correctness of the Lean kernel and Mathlib; (2) the faithfulness of the Lean definitions to the intended geometry; (3) the correctness of drat-trim; (4) file identity established through SHA-256, and the correctness of the Python transcriptions and file-comparison programs; and (5) for the lower bounds, the correctness of the geometric witness checkers.

3 Coronas and Heesch numbers on three grids

We follow Kaplan [10, Sections 2.1 and 4]. Points of $\mathbb{Z}^2$ index cells. A *grid* specifies which points are cells, when two cells *touch* (share at least one point of the plane), when they share an edge, which affine maps $x \mapsto Mx + w$ are allowed congruences, and what a *pinch vertex* is. Table 2 gives the three grids. On the hexagonal grid the point (x, y) is the centre $xv + yw$ of a regular hexagon, with $v = (1, 0)$ and $w = (1/2, \sqrt{3}/2)$. The triangular grid uses the same coordinates: a point is a cell (a triangle with that centroid) if $x \equiv y \equiv 0$ or $x \equiv y \equiv 1 \pmod 3$; points with $x \equiv y \equiv 2 \pmod 3$ are vertices. The black cell (x, y) has vertices $(x - 1, y + 2)$, $(x - 1, y - 1)$, $(x + 2, y - 1)$, and the grey cell (x, y) has vertices $(x + 1, y - 2)$, $(x + 1, y + 1)$, $(x - 2, y + 1)$. The twelve hexagonal orientations are $A^i B^j$ $(0 \le i < 6$, $j \in \{0, 1\})$ with

$$A = \begin{pmatrix} 0 & -1 \\ 1 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 1 & 1 \\ 0 & -1 \end{pmatrix},$$

acting by $(x, y) \mapsto (ax + by,\ cx + dy)$ for the matrix $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$.

A *shape* S is a finite set of cells, a *copy* of S is its image under an allowed congruence, and $\mathrm{Halo}(X)$ is the set of cells not in X that touch a cell of X. A finite set P of cells has a *hole* if some cell outside P lies in a finite component of the complement of P under edge adjacency. We call P a *disc* if (a) P is connected under edge adjacency, (b) P has no hole, and (c) P has no pinch vertex. For finite unions of cells these three conditions describe a closed topological disc; the formal statement below uses the combinatorial conditions themselves.

	Square	Hexagonal	Triangular
Cells	all of $\mathbb{Z}^2$	all of $\mathbb{Z}^2$	$x \equiv y \equiv 0$ or $x \equiv y \equiv 1 \pmod 3$
Touch	8 neighbours N_8	6 offsets $(\pm 1, 0)$, $(0, \pm 1)$, $\pm(1, -1)$	share ≥ 1 vertex (12 neighbours)
Share an edge	4 neighbours N_4	same as touch	share 2 vertices (3 neighbours)
Orientations	4 rotations, 4 reflections	$A^i B^j$ (12)	$A^i B^j$ (12)
Translations w	$\mathbb{Z}^2$	$\mathbb{Z}^2$	$M(2,2) + w \equiv (2,2) \pmod 3$
Distance d	$\max(\lvert a\rvert, \lvert b\rvert)$	$\max(\lvert a\rvert, \lvert b\rvert, \lvert a+b\rvert)$	0 for the same cell; otherwise $1 +$ min of the hexagonal distance of $(u - u')/3$ over vertices u, u' of the two cells
Pinch vertex	two diagonal cells of four	none	incident cells form ≥ 2 runs around a vertex
Scan box for (F)	$\pm K$	$\pm K$	$\pm 4K$

Table 2: The three grids. (a, b) is the difference of two cells; the distance from a set T_0 is the minimum over its cells. On the triangular grid the translation condition says that vertices are mapped to vertices; it selects one residue class of w modulo $3\mathbb{Z}^2$ for each matrix.

Definition 1 (Coronas)**.** Let $k \geq 0$. A *k-corona* of S is a finite set of copies of S, each with a layer number in $\{0, \ldots, k\}$, such that, writing P_j for the union of the copies of layer at most j,

(P0) the only copy of layer 0 is S itself;

(P1) distinct copies are disjoint;

(P2) $\mathrm{Halo}(P_j) \subseteq P_{j+1}$ for $j < k$;

(P3) every copy of layer $j \geq 1$ has a cell in $\mathrm{Halo}(P_{j-1})$.

It is an *H_{h}-corona* if $P_0, \ldots, P_{k-1}$ are discs, and an *H_{c}-corona* if $P_0, \ldots, P_k$ are discs. The numbers $H_{\mathrm{c}}(S)$ and $H_{\mathrm{h}}(S)$ are the largest k for which an H_{c}-corona, respectively an H_{h}-corona, with k layers exists.

All copies lie on the fixed grid, as in Kaplan's SAT setting [10, Section 3]. Reported Heesch values concern hole-free disc-shaped polyforms. If the central shape has a hole, the P_0 condition excludes every positive-layer H_{h}-corona and even a zero-layer H_{c}-corona; such shapes are omitted from Kaplan's files used here.

Lemma 2. (i) *If S has an H_{h}-corona (H_{c}-corona) with k layers, it has one with k' layers for every $k' \leq k$.* (ii) $H_{\mathrm{h}}(S) - 1 \leq H_{\mathrm{c}}(S) \leq H_{\mathrm{h}}(S)$.

Proof. (i) Keep the copies of layer at most k'. The sets $P_0, \ldots, P_{k'}$ do not change, so (P0)–(P3) and the disc conditions for $j \leq k'$ are inherited. (ii) An H_{c}-corona is an H_{h}-corona. Truncating an H_{h}-corona with k layers to $k - 1$ layers gives an H_{c}-corona, since $P_0, \ldots, P_{k-1}$ are discs. □

Consequently, the nonexistence of an H_{h}-corona with K layers gives $H_{\mathrm{c}} \leq H_{\mathrm{h}} \leq K - 1$, and the nonexistence of an H_{c}-corona with K layers gives $H_{\mathrm{c}} \leq K - 1$.

4 The SAT encoding

The input is a shape S, a grid, and a number of layers $K \geq 1$. Write $T_0 = S$ and let $d(c)$ be the distance of a cell c from T_0 in the touching graph (Table 2).

Lemma 3 (Distance bound)**.** *In a k-corona, every cell c with $d(c) \leq j \leq k$ lies in P_j. Hence every cell of a copy of layer j has $d \geq j$.*

Proof. By induction on j, using $P_j \supseteq P_{j-1} \cup \mathrm{Halo}(P_{j-1})$, which is (P2). A cell with $d < j$ then lies in P_{j-1}, so by (P1) it is not in a copy of layer j. □

Pruning. Let $R_0 = T_0$. For $j = 1, \dots, K$, a copy p of S disjoint from T_0 is a candidate at layer j if its halo meets R_{j-1}. A candidate is given the interval of layers $\Lambda(p) = [\mathrm{lo}(p), \mathrm{hi}(p)]$, where $\mathrm{lo}(p)$ is the first j at which it qualifies and $\mathrm{hi}(p) = \min(K, \min_{c \in p} d(c))$, and R_j is R_{j-1} together with the cells of all candidates allowed at layer j. Then, for $j < K$, a pair (p, j) is kept only if $\mathrm{Halo}(p) \setminus T_0 \subseteq R_{j+1}$, the sets R_j are recomputed from the remaining pairs, and the touching and halo tests are repeated until nothing changes. By Lemma 3, (P2) and (P3), every copy of a genuine corona survives with its layer: the cells it touches lie in R_{j-1} and the copies covering its halo lie in R_{j+1}.

Variables. $t_{p,j}$ for $j \in \Lambda(p)$ (copy p is used at layer j); $y_{c,j}$ for $1 \le j \le K$ and $c \in R_j \setminus T_0$ (cell c is covered by layers $\le j$); auxiliary variables of the at-most-one counters; and, if needed, one fresh variable b.

Clauses.

(F) $y_{c,d(c)}$ for every cell with $1 \le d(c) \le K$ (Lemma 3). If some such c is not in $R_{d(c)}$, the clauses b and $\neg b$ are added instead of the missing unit.

(Y1) $y_{c,j-1} \to y_{c,j}$.

(Y2) $y_{c,j} \to y_{c,j-1} \vee \bigvee\{t_{p,j} : c \in p\}$, the first disjunct omitted when $y_{c,j-1}$ does not exist.

(B) $t_{p,j} \to y_{c,j}$ for $c \in p$.

(C) For each cell c, at most one of the variables $t_{p,j}$ with $c \in p$ is true, by Sinz's sequential counter [17]: for inputs $x_1, \dots, x_m$ and auxiliary $s_1, \dots, s_{m-1}$, the clauses $x_1 \to s_1$; $x_a \to s_a$, $s_{a-1} \to s_a$, $x_a \to \neg s_{a-1}$ for $1 < a < m$; and $x_m \to \neg s_{m-1}$. This gives (P1) and a single layer per copy.

(D) For $j < K$: $t_{p,j} \to y_{c,j+1}$ for $c \in \mathrm{Halo}(p) \setminus T_0$; if $y_{c,j+1}$ does not exist, the clause is $\neg t_{p,j}$. This gives (P2).

(E) For $j \ge 2$ and p not touching T_0: $t_{p,j} \to \bigvee\{y_{c,j-1} : c \in \mathrm{Halo}(p) \setminus T_0\}$. This gives (P3); for $j = 1$ it holds by construction.

The order in which the inputs of each counter are listed is a free choice. We write ord for this choice and $F_{\mathrm{ord}}(S, K, \mathcal{B})$ for the resulting CNF with a list $\mathcal{B}$ of hole blocks.

Hole blocks. A satisfying assignment may describe a patch whose layer P_j has a hole, which an H_{h}-corona forbids for $j \le K - 1$ and an H_{c}-corona for $j \le K$. A *block* (j, H, E) consists of a level j, a set of cells H and a list E of copies with layers. It is *valid* for a maximal level L if $1 \le j \le L$, H is nonempty and consists of cells, every copy in E has layer at most j, and every edge neighbour of H outside H lies in a copy of E. It adds the clause

$$\bigvee_{(p,l) \in E} \neg t_{p,l} \;\vee\; \bigvee_{c \in H} y_{c,j},$$

in which the entry $(0, T_0)$ contributes no literal, and each $y_{c,j}$ that does not exist is omitted. Such a cell cannot belong to P_j. The block is skipped if H meets T_0 or if some other entry of E is not a variable. The clause can be empty, which is still sound for a valid block. Kaplan's hole clauses [10, Section 3.2] contain only the negative literals; the positive literals added here keep the assignments in which other copies fill H, so these clauses are weaker. If every copy in E is used and no cell of H is covered by layers $\le j$, then H is cut off from the rest of the complement of P_j, so P_j has a hole. On the triangular grid the requirement that H consist of cells matters: a non-cell point has no edge neighbours, so without it a block on such a point would produce an empty clause.

Theorem 4 (Soundness)**.** *Let the grid be square, hexagonal or triangular, let S be a finite set of points, $K \ge 1$, let* ord *list each counter's inputs in some permutation, and let every block in $\mathcal{B}$ be valid for maximal level $K - 1$ (resp. K). If $F_{\mathrm{ord}}(S, K, \mathcal{B})$ is unsatisfiable, then S has no H_{h}-corona (resp. H_{c}-corona) with K layers.*

Proof outline. Given such a corona, set $t_{p,j}$ true when p is a copy of layer j, $y_{c,j}$ true when $c \in P_j$, and each counter variable s_a true when one of the first a inputs is. Pruning keeps every genuine copy at its layer, so the variables exist and (F) needs no fresh variable. Clauses (Y1), (Y2), (B) and (D) follow from the definitions and (P2), (C) from (P1), (E) from (P3), and each block clause from the disc condition of P_j. The formal proof is described in Section 5. □

With an empty block list, the same assignment in the proof outline satisfies the CNF for every patch obeying (P0)–(P3), without any disc conditions: no block clause remains to use them. Thus UNSAT excludes all such patches. This is a prose consequence of the proof, not one of the six formal statements in Lean (Table 1). The value computation solves $K = 1, 2, \ldots$ without hole conditions on the outer layer until the first unsatisfiable K, which gives $H_\mathrm{h} = K - 1$; when a solution has a hole in an inner layer, a block is added and the problem is solved again. If the witness for H_h has a hole in its outer layer, the problem with $K = H_\mathrm{h}$ and the H_c condition decides between $H_\mathrm{c} = H_\mathrm{h}$ and $H_\mathrm{c} = H_\mathrm{h} - 1$ (Lemma 2). Hole blocks can occur in intermediate SAT calls during the value search; the final upper-bound certificates use blocks only for H_c, as shown in Table 3.

5 The Lean formalization

The formal statement imports only Mathlib [23] and was written before the proof, directly from the mathematical description above. A structure `Grid` carries the fields of Table 2: a predicate `isCell`, the relations `Touch` and `EdgeAdj`, a predicate `Pinch`, the list of orientation matrices, the translation test `transOk`, and three computable helpers (the list of touching cells, the distance d, and the box scanned by (F)). The three grids are the values `squareGrid`, `hexGrid` and `triGrid`; the twelve hexagonal matrices are computed inside the statement as products $A^i B^j$. Copies, halos, holes, discs and the predicates `IsCoronaHh` and `IsCoronaHc` are defined once for an arbitrary grid, following Definition 1. A hole cell is required to satisfy `isCell`; otherwise every non-cell point of the triangular grid would be a finite component of the complement and every patch would have a hole. The pruning and the CNF are computable functions `candidates` and `cnfOrd`, a CNF being a list of lists of integer literals, with literal 0 interpreted as false. The six theorems `hh_sound_square`, `hc_sound_square`, `hh_sound_hex`, `hc_sound_hex`, `hh_sound_tri` and `hc_sound_tri` state Theorem 4. The triangular H_h case reads (in ASCII notation; the source uses the corresponding Unicode symbols):

```
theorem hh_sound_tri (ord : Cell -> List Int -> List Int)
    (hord : forall c g, (ord c g).Perm g)
    (S : List Cell) (K : Nat) (hK : 1 <= K) (blocks : List HoleBlock)
    (hblocks : forall b, b ∈ blocks -> b.Valid triGrid (K - 1))
    (hunsat : Not (Satisfiable (cnfOrd triGrid ord S K blocks))) :
    Not (exists tiles : Tiles, IsCoronaHh triGrid S.toFinset K tiles)
```

Here `Satisfiable F` means that some assignment $\sigma : \mathbb{N} \to \{\text{true}, \text{false}\}$ makes a literal of every clause true, and `HoleBlock.Valid` is the validity condition of Section 4.

Proof. The proof constructs the satisfying assignment of the proof outline of Theorem 4. All grid-dependent facts are collected in nine properties (touching agrees with the neighbour list and is symmetric, touching cells are cells, edge adjacency is symmetric, four properties of the distance, and allowed congruences map non-cells to non-cells), proved separately for each grid. Lemma 3 then follows for every grid from the property that the distance decreases by one along some touching step. On the triangular grid this step property is the main geometric lemma: from a minimal pair of vertices one moves along an edge of the vertex lattice, and the triangle on that edge touches the current cell and is one step closer. The survival of genuine copies under pruning is proved by an invariant of each refinement round; it uses neither that the iteration reaches a fixed point nor that its bound on the number of rounds suffices. The proof files have 1,837 lines.

Checks. Comparator, in paranoid mode, checks that the proof proves exactly the six statements of the statement file and replays them in six kernel runs using five implementations: the Lean kernel in paranoid and default mode, lean4lean, nanoda, con-leche and con-ron; all six runs accept [24, 25, 26]. `#print axioms` reports only `propext`, `Classical.choice` and `Quot.sound`, and the files contain no `sorry`, `native_decide` or new axiom. The environment is Lean 4.35.0 [22] with Mathlib v4.35.0 [23]. A further theorem, outside the Comparator statement, shows that `cnfOrd squareGrid` equals an earlier square-only definition for all arguments, so results established for the earlier definition carry over.

Reading of the statement. An independent AI reader compared the statement with the mathematical description and with Sections 2.1 and 4 of [10], before seeing any proof. It checked the hexagonal and

triangular cells, neighbours, orientations and translation classes, compared both distance formulas with breadth-first search, checked the pinch test on all 64 occupancy patterns around a triangular vertex, and confirmed on small examples that the definitions are not vacuous. Its verdict was "faithful with caveats". The caveats are part of the scope: the theorems accept any finite set of points as S, including sets that are not polyforms (the theorems remain true for such inputs but then say nothing about polyforms); copies are placed on the fixed grid; and the link between a checked DIMACS file and `cnfOrd` lies outside the statement.

6 Certificates

Shapes. The shapes and reference values are read from Kaplan's published files [10], two lines per shape (cell coordinates and the values H_c, H_h). The 22 files that contain the selected shapes, and the files searched to complete the selection, are identified by their SHA-256 hashes in the verification record. For each grid, Kaplan publishes all hole-free non-tilers up to a size and, for larger sizes, only the shapes with H_h above a threshold. Our selection is listed in Table 4.

Lower bounds. For each shape we give a witness for H_h and one for H_c, combined into a single witness when the same patch serves both (1,354 shapes), in total 6,574 witnesses. A witness lists every copy with its layer and absolute cells. Two separately written checkers, neither of which uses the encoding, test congruence of every copy, (P0)–(P3), the disc conditions (a)–(c) for the layers required by H_c or H_h, and that k equals the claimed value. Both accept all 6,574 witnesses.

Upper bounds. For each shape, one certificate states that there is no H_h-corona with $K = H_h + 1$ layers; all 3,964 of these use an empty block list, so the prose consequence of the soundness proof excludes every K-layer patch satisfying (P0)–(P3), even without disc conditions. For the 1,055 shapes with $H_c < H_h$, a second certificate states that there is no H_c-corona with $K = H_c + 1$ layers; these carry 13,287 hole blocks in total. Each certificate records the CNF (SHA-256, size, numbers of variables and clauses) and a proof of unsatisfiability. The proofs were produced by CaDiCaL 1.5.3 [28] (4,998 binary DRAT proofs), Glucose 4.1 [29] (16 DRUP proofs) and Lingeling [30] (5 text DRAT proofs), all through python-sat 1.9.dev17 [27], and all 5,019 are accepted by drat-trim [21]. The CNFs and proofs are not archived: the largest single proof has 7,175,887,914 bytes. The CNFs are regenerated deterministically: all 5,019 agree with the recorded hashes and sizes. Proofs are regenerated by rerunning the recorded solver. Proof-hash reproduction has been tested in four large cases: five-layer H_h bounds for two 15-cell polyhexes, a 13-cell polyhex and a 20-cell polyiamond. Each regenerated proof agrees with the recorded hash and is accepted by drat-trim, with at most 3.01 GB of memory.

Second encoding. A second encoding, written from the mathematical description of the encoding without access to the first implementation and solved with the same SAT solvers through python-sat, recomputes all bounds. Its encoding code is independent; the mathematical specification and SAT solvers are shared. It agrees with the first encoding and with Kaplan on H_c and H_h for all 3,964 shapes; the 6,574 witnesses, read as assignments, satisfy all of its clauses; and all 5,019 upper-bound instances are unsatisfiable. This second computation did not produce proof logs; it is an independent check of the values, not a second certificate.

			Certificates		Accepted by		Largest CNF
Grid	Shapes	Witnesses	H_h	H_c	drat-trim	Blocks	(clauses)
Square	1,616	2,766	1,616	460	2,076	7,267	2,926,850
Hexagonal	437	808	437	100	537	2,126	6,588,816
Triangular	1,911	3,000	1,911	495	2,406	3,894	5,869,683
Total	3,964	6,574	3,964	1,055	5,019	13,287	

Table 3: Witnesses and upper-bound certificates. Blocks occur only in the H_c certificates. The largest CNFs are a four-layer bound for a 17-omino, and five-layer bounds for a 15-hex and a 20-iamond.

Grid	Cells (selection)	Shapes	Number of shapes with $(H_c, H_h) =$								
			(0,0)	(0,1)	(1,1)	(1,2)	(2,2)	(2,3)	(3,3)	(3,4)	(4,4)
Square	7 (all)	3		1	2						
	8 (all)	20		6	13	1					
	9 (all)	198	36	39	118	4	1				
	10 (all)	1,390	355	392	628	14	1				
	11–19 ($H_h = 3$)	5						3	2		
Hexagonal	6 (all)	4			3		1				
	7 (all)	37	4	1	18	7	5	1	1		
	8 (all)	381	37	33	220	44	40	4	3		
	9–16 ($H_h = 4$)	15								10	5
Triangular	7 (all)	1			1						
	9 (all)	20	7	4	9						
	10 (all)	103	33	11	48	7	3			1	
	11 (all)	594	117	119	327	19	11		1		
	12 (all)	1,192	495	331	361	3	1		1		
	13–24 ($H_h = 4$)	1									1
Total		3,964	1,084	937	1,748	99	63	8	8	11	6

Table 4: The certified Heesch numbers. "All" means every shape in Kaplan's file of hole-free non-tilers with that number of cells. The polyominoes with $H_h = 3$ have 11, 13, 17, 17 and 17 cells (one with $(2, 3)$ each for 11 and 13 cells; $(2, 3)$, $(3, 3)$, $(3, 3)$ for 17 cells). The polyhexes with $H_h = 4$ have 10 (1), 11 (6), 12 (2), 13 (2), 15 (2) and 16 (2) cells. The polyiamonds with $H_h = 4$ are a 10-iamond with $(H_c, H_h) = (3, 4)$ and a 20-iamond with $(4, 4)$. There are no non-tiling 8-iamonds in Kaplan's census (his Table 5). Kaplan's files for $n \geq 11$ cells (square), $n \geq 9$ (hexagonal) and $n \geq 13$ (triangular) list only shapes above a threshold of H_h; the selected shapes are all shapes in these files with the stated H_h, the largest value occurring in each group of files.

7 From the checked files to the Lean CNF

Theorem 4 is about `cnfOrd`; drat-trim checked DIMACS files written by a separate program. Two CNFs F and F' are *equivalent up to renaming* if there is a bijection π of the variables such that applying π (without changing signs) to F gives the same multiset of clauses as F', each clause compared as a multiset of literals. Such F and F' are equisatisfiable. For each of the 5,019 certificates we compared the checked file with `cnfOrd G ord S K blocks` for the recorded grid, shape, K and blocks, as follows.

1. The checked file is regenerated and its SHA-256 hash and size are compared with the certificate record (all 5,019 agree).

2. The Lean definition of `cnfOrd` is transcribed into Python without reading the source of the generating program. The transcription uses two Python transcriptions, one for the square grid and one for the hexagonal and triangular grids. They are validated against Lean's own evaluation: 64 outputs (30 square and 34 hexagonal/triangular, including hole blocks, the fresh variable b, and default and reversed counter orders) agree byte for byte. For the square grid, the comparison is with the earlier square-only definition; the Lean equality theorem outside the Comparator statement identifies it with `cnfOrd squareGrid`.

3. From the checked file, the input order of every at-most-one counter and a bijection of variables are recovered; the recovered orders are checked to be permutations.

4. The checked file, renamed by the bijection, and the transcription with the recovered orders are compared clause by clause as multisets, read by two different DIMACS parsers, without merging duplicates, simplifying or flipping signs.

5. Every block is checked against `HoleBlock.Valid`; on the triangular grid all 6,922 hole points are cells.

The two programs differ in where they number the fresh variable b (after the cell variables, or after the counter variables) and in the order of literals within clauses; both differences are absorbed by the bijection and by comparing clauses as multisets. The comparison was performed for all 5,019 CNFs (Table 5), with no mismatch and no undecided case. No compared clause contains literal 0; the DIMACS clause terminators are removed before comparing literals. This distinguishes the file convention from Lean's false-literal convention.

Grid	CNFs	Variables	Clauses	Blocks	CNFs by K ($K = 1, 2, 3, 4, 5$)
Square	2,076	45,711,523	178,454,670	7,267	829, 1,218, 24, 5, 0
Hexagonal	537	39,687,752	168,537,425	2,126	75, 326, 102, 19, 15
Triangular	2,406	48,014,519	191,702,313	3,894	1,117, 1,240, 44, 3, 2
Total	5,019	133,413,794	538,694,408	13,287	

Table 5: The file comparison. All 5,019 checked CNFs are equivalent up to renaming to `cnfOrd` with the recovered counter order, and all blocks are valid. Variables and clauses are summed over the CNFs.

Corollary 5. *Assume (1) the correctness of the Lean kernel and Mathlib; (2) the faithfulness of the Lean definitions to the intended geometry; (3) the correctness of drat-trim; (4) file identity established through SHA-256, and the correctness of the Python transcriptions and file-comparison programs; and (5) for the lower bounds, the correctness of the geometric witness checkers. Then for each shape in Table 4, there is no H_{h}-corona with $H_{\mathrm{h}} + 1$ layers and no H_{c}-corona with $H_{\mathrm{c}} + 1$ layers, so the upper bounds hold; the witnesses give the lower bounds.*

Proof. By the comparison, each checked CNF is equisatisfiable with `cnfOrd` for a permutation order and valid blocks, so Theorem 4 applies; Lemma 2 turns the nonexistence of K-layer coronas into the bound $K - 1$. For a shape with $H_{\mathrm{c}} = H_{\mathrm{h}}$, the H_{h} certificate also bounds H_{c}. □

8 Results

Table 4 lists the selection and the distribution of the certified values. In every one of the 3,964 shapes, the certified pair $(H_{\mathrm{c}}, H_{\mathrm{h}})$ equals the pair in Kaplan's file. Within the selection, every shape satisfies $H_{\mathrm{c}} \in \{H_{\mathrm{h}} - 1, H_{\mathrm{h}}\}$, as Lemma 2 requires. The largest values are $H_{\mathrm{h}} = 3$ for polyominoes and $H_{\mathrm{h}} = 4$ for polyhexes and polyiamonds; the five polyhexes with $H_{\mathrm{c}} = H_{\mathrm{h}} = 4$ (one 11-hex, one 13-hex, two 15-hexes and one 16-hex) and the 20-iamond with $H_{\mathrm{c}} = H_{\mathrm{h}} = 4$ have the largest certified hole-free values. The other polyiamond with $H_{\mathrm{h}} = 4$ is the 10-iamond with $H_{\mathrm{c}} = 3$.

9 Counting maximal coronas

A *maximal H_{h}-corona* of S is an H_{h}-corona of S with exactly $k = H_{\mathrm{h}}(S)$ layers, and a *maximal H_{c}-corona* is an H_{c}-corona with the same number k of layers; "maximal" refers to the number of layers, not to inclusion. The 22 shapes in Table 6 attain the largest H_{h} in each grid range selected here: five polyominoes with $H_{\mathrm{h}} = 3$, fifteen polyhexes with $H_{\mathrm{h}} = 4$, and two polyiamonds with $H_{\mathrm{h}} = 4$. The polyhex range stops at 16 cells: Kaplan's published Table 4 has an additional 17-cell example, which is outside our inputs. For each shape we count H_{h}-coronas at $k = H_{\mathrm{h}}(S)$; for the eight shapes with $H_{\mathrm{c}} = H_{\mathrm{h}}$ we also count H_{c}-coronas at the same k.

9.1 Objects, symmetry and prefixes

A configuration is a finite set of pairs (j, p), where p is the cell set of a copy and j its layer. The counting conventions are:

1. Two configurations are identified exactly when a grid congruence fixing the central cell set S maps one to the other. This group is the central stabilizer $G(S)$.

2. Copies are cell sets: their ordering, multiple congruences giving the same copy, and assignments to the SAT auxiliary variables are not counted.

3. Layer labels are retained. They can also be recovered from the set of copies: once P_{j-1} is known, every copy outside it that touches it must have layer j, since a higher-layer copy would overlap the halo already covered by P_j. Induction recovers all layers.

4. Extra copies beyond a corona are not permitted. Each copy in layer j must touch P_{j-1}, as required by (P3).

The raw count distinguishes all coordinates; the orbit count divides only by the action of $G(S)$. A finite calculation gives $|G(S)| = 1$ for all 22 central shapes, so the two counts coincide.

A prefix of depth j is the restriction of a maximal k-corona to layers at most j. Write N_j for the number of distinct such prefixes. These are precisely the prefixes that extend to k layers, not all j-coronas. An *inner part* is a prefix of depth $k-1$.

Proposition 6 (Minimality of the counted configurations)**.** *No two distinct k-coronas satisfying (P0)–(P3), considered as sets of layered copies, contain one another. The same holds for their prefixes of a fixed depth when both prefixes extend to k layers.*

Proof. Suppose $Q \subsetneq Q'$ and take an extra copy of Q' of smallest layer i. Both configurations have the same P_{i-1}. By (P3) that copy contains a cell of $\mathrm{Halo}(P_{i-1})$, already covered by a layer-i copy of Q by (P2). It therefore overlaps a different copy of Q', contradicting (P1). The same argument works for a prefix, since (P2) through the preceding layer is inherited from its extension. □

9.2 Enumeration and completion counts

The computation separates inner parts from the last layer. At $K = k$, a SAT solver enumerates projections onto the inner placement variables. Every solution found is a complete k-layer configuration and is checked by the geometric witness checker. For the inner layers $P_1, \ldots, P_{k-1}$ the disc conditions reduce to the absence of holes. By (P2), P_j contains $P_{j-1} \cup \mathrm{Halo}(P_{j-1})$; a cell touching P_{j-1} is joined to it by edge-adjacent cells around a common vertex, all in this union, and every copy of layer j is edge-connected and meets $\mathrm{Halo}(P_{j-1})$. By induction from $P_0 = S$, each P_j is edge-connected. In an edge-connected set, a pinch vertex of the square or triangular grid encloses a cell of the complement and so produces a hole, and the hexagonal grid has no pinch vertices. When a solution has a hole in an inner layer, a hole clause is added as in Section 4. A blocking clause containing the negations of the selected inner placements removes just that prefix, by Proposition 6; this is a prose argument, not an extension of the Lean theorem. A second implementation instead blocks the complete signed assignment of all inner placement variables, so it does not depend on that minimality argument.

For a fixed inner part with union $U = P_{k-1}$, an outer layer is an exact cover of $\mathrm{Halo}(U)$ by copies disjoint from U and from each other. Disjointness is tested on the whole cell sets, including cells outside the halo. Each selected copy covers at least one halo cell, so (P3) excludes surplus copies. For H_c the completed union must also be a disc. Summing the number of outer completions over the distinct inner parts gives N_k.

Two separate implementations enumerate the inner projections and count the outer completions, sharing the CNF encoding, hole-clause generator, grid definitions and geometric witness checker. The second independently generates outer candidates and uses a different projected blocking rule. For H_c it uses direct disc tests on the square grid. On the hexagonal grid the completed union is edge-connected and has no pinch vertex, so it is a disc exactly when its Euler characteristic is 1; on the triangular grid the same test is combined with boundary conditions that exclude pinch vertices. The first uses direct geometric hole pruning with memoization. Agreement is checked per inner configuration in cell coordinates, not just in the final sum.

In 28 of 30 combinations, the prefix sets, all prefix counts, every inner part's completion count and the total agree. Both implementations have drat-trim-checked exhaustion proofs for those inner enumerations. Such a proof certifies that the final enumeration CNF, consisting of the base clauses, the inner hole clauses and all blocking clauses, is unsatisfiable. The enumeration CNFs are produced by the same generator as the certified upper-bound CNFs, but they are covered neither by the Lean theorem, which has no blocking clauses, nor by the file comparison of Section 7, which concerns the 5,019 upper-bound CNFs.

That no further extendable inner part exists therefore also rests on the shared generator, the validity of its hole clauses and, for the first implementation, Proposition 6. The outer completion counts themselves have no proof certificate and are not formalized in Lean. They are exact finite counts corroborated by the separate implementations. Further checks enumerate complete k-coronas directly by SAT, without separating the outer layer: nine combinations in the first implementation, each ending in a drat-trim-checked unsatisfiable call, and six in the second. The second implementation also recounts the outer layers of 12 inner parts, in eight combinations, by backtracking without memoization. All these checks agree with Table 6.

For 15-hex #315 both implementations enumerate the same 979 H_{h} and 79 H_{c} inner parts and the same shallower prefixes, but neither has a checked DRAT exhaustion proof for them. Its totals in Table 6 are from the second counter only. The first implementation completed the outer counts of 26 of the 79 H_{c} inner parts before stopping. Of these, 22 agree with the second implementation; the other four were flagged as invalid by the first implementation's own check on its memoized hole tests. It did not finish the H_{h} outer count; the three per-inner-part values it reported agree with the second implementation. These counting exhaustion checks are distinct from the checked five-layer upper-bound certificate for this shape in Table 3.

Table 6: Maximal-corona counts. Shape identifiers are zero-based row indices in Kaplan's indicated size/threshold files. Here $k = H_{\mathrm{h}}(S)$; N_j counts prefixes extendable to a k-corona of the indicated variant, and the last column is N_k. A dash marks $j = k$, already in the last column. Every central stabilizer is trivial, so raw and orbit counts coincide. All entries agree between the two implementations except the † totals, available from one only; their prefix counts agree but have no checked DRAT exhaustion proof.

Shape	$(H_{\mathrm{c}}, H_{\mathrm{h}})$	Variant	N_1	N_2	N_3	N_k
11-omino #2	$(2,3)$	H_{h}	1	1	–	1,250
13-omino #50	$(2,3)$	H_{h}	1	1	–	594
17-omino #43	$(3,3)$	H_{h}	1	1	–	430
17-omino #43	$(3,3)$	H_{c}	1	1	–	28
17-omino #192	$(3,3)$	H_{h}	2	4	–	1,168
17-omino #192	$(3,3)$	H_{c}	1	1	–	2
17-omino #521	$(2,3)$	H_{h}	1	2	–	20,160
10-hex #13	$(3,4)$	H_{h}	3	3	9	768,724
11-hex #18	$(4,4)$	H_{h}	1	1	5	20,610
11-hex #18	$(4,4)$	H_{c}	1	1	2	57
11-hex #20	$(3,4)$	H_{h}	1	1	2	45,923,328
11-hex #30	$(3,4)$	H_{h}	3	6	51	60,618,972
11-hex #32	$(3,4)$	H_{h}	2	2	7	7,030,233,087
11-hex #37	$(3,4)$	H_{h}	1	1	1	1,581,552
11-hex #59	$(3,4)$	H_{h}	1	1	2	2,303,840
12-hex #5	$(3,4)$	H_{h}	4	12	480	6,566,213,760
12-hex #21	$(3,4)$	H_{h}	1	1	1	56
13-hex #19	$(4,4)$	H_{h}	1	1	3	91,614,336
13-hex #19	$(4,4)$	H_{c}	1	1	1	98,304
13-hex #38	$(3,4)$	H_{h}	3	3	7	21,102,048
15-hex #278	$(4,4)$	H_{h}	12	18	126	261,662,826
15-hex #278	$(4,4)$	H_{c}	3	3	11	543,708
15-hex #315	$(4,4)$	H_{h}	10	29	979	13,031,023,098,920†
15-hex #315	$(4,4)$	H_{c}	6	14	79	269,859,001†
16-hex #2	$(4,4)$	H_{h}	1	1	1	24,240
16-hex #2	$(4,4)$	H_{c}	1	1	1	880
16-hex #72	$(3,4)$	H_{h}	2	2	5	300,583,329
10-iamond #51	$(3,4)$	H_{h}	1	12	448	342,830,592
20-iamond #130	$(4,4)$	H_{h}	1	6	46	932,565,708
20-iamond #130	$(4,4)$	H_{c}	1	1	2	2,418,624

9.3 Forced inner parts and non-forced outer layers

In every fully compared combination, each fixed extendable inner part has at least two outer completions. The smallest completion count is two for 17-omino #192 in the H_c variant. Thus no outer layer is forced in those 28 combinations. For the two remaining combinations, the single completed counter reports minimum completion counts 1,152 (H_h) and 72 (H_c).

Exactly six fully compared H_h cases have a unique inner part: 11-omino #2, 13-omino #50, 17-omino #43, 11-hex #37, 12-hex #21 and 16-hex #2. Exactly four fully compared H_c cases do: 17-omino #43, 17-omino #192, 13-hex #19 and 16-hex #2. The matching cell-coordinate sets identify the unique prefix itself, not just its count; Figure 1 shows the one for 11-omino #2. In particular, 12-hex #21 has one inner part and 56 outer layers; 16-hex #2 has one inner part in either variant but 24,240 H_h completions and 880 H_c completions. The large total for 11-hex #32 is concentrated in one of its seven inner parts, with 7,029,620,442 completions.

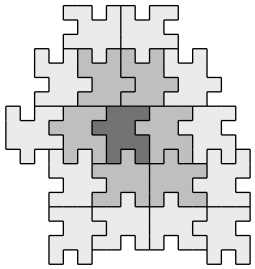


Figure 1: The unique inner part of 11-omino #2 ($k = H_h = 3$), drawn from the recorded cell coordinates: the central shape (dark), its six copies of layer 1 (medium) and twelve copies of layer 2 (light). Both implementations record exactly this configuration. It has 1,250 outer completions.

9.4 Comparison with public enumeration reports

Enumerating coronas and studying forced configurations have clear precedents. Kaplan's heesch-sat software [12] enumerates the one-layer surrounds of a shape, separating those with and without holes, and contains a routine that counts hole-free coronas by successively excluding SAT solutions. Mann [15] discusses generating all first and second coronas of a shape. Kaplan [10, p. 170 of the journal version] points to the forced groupings of tiles described by Mann and Thomas [16]. Public reports of the Heesch challenge [31] enumerate coronas of several shapes counted here. To compare like quantities, separate calculations with the first implementation count all one- and two-layer H_c-coronas, without requiring extension to four layers; each enumeration ends with a drat-trim-checked unsatisfiable call, and these counts were not repeated by the second implementation. For 16-hex #2 the counts 251 and 1,449 match discussion #32; the unique second-layer prefix extendable to four layers also matches the reported count. For 11-hex #18 the one-layer count 793 matches #12. Shape coordinates have not been compared with those reports, so equal reported counts are not treated as a proof of input identity.

Discussion #94 reports 1,027 complete four-layer patches of the 20-iamond, whereas our H_c count is 2,418,624. Its counted object, equivalence relation and treatment of unfinished branches have not been aligned with ours. The discrepancy therefore does not identify an error on either side. We provide counts under the explicit conventions above, without a priority claim for counting coronas.

10 A restricted search for finite-$H_h = 4$ polyominoes

Kaplan's published search through 19 cells gives largest finite $H_h = 3$ for unmarked polyominoes [10]. We examine a fixed collection of 3,442 hole-free, edge-connected polyominoes with 20–28 cells. Candidates are normalized under translation, rotation and reflection and arise from five biased construction families: anisotropic integer enlargement of the five polyominoes with $H_h = 3$ in Table 4 (4); repeating one row or column of one of these five shapes one to three times (82); two successive such repetitions (1,223); cell additions to the same five shapes until 20 cells (612); and one-cell additions to the 767 shapes of Kaplan's file `19omino_2up` (1,521). That file lists the 19-cell polyominoes with $H_h \geq 2$, all of which have $H_h = 2$;

it is not a list of all 19-cell polyominoes. Duplicates are assigned to the earliest family. The final family selects at most two shapes per parent in a fixed hash order from 17,302 generated free shapes; the full generated family is not screened.

The screening tests $K = 3$ for H_{h}, with a $K = 1$ pretest for part of the collection. By Lemma 2, either UNSAT result excludes $H_{\mathrm{h}} = 4$. Satisfying patches are checked geometrically. Explicit periodic tilings are then sought before deeper SAT tests. A tiling certificate places congruent copies in an integer-lattice fundamental domain and checks complete coverage and non-overlap, including all translates. The retained outcomes are in Table 7. The two UNSAT columns describe the actual screening stages, not exact Heesch values.

Cells	$K = 1$ UNSAT	$K = 3$ UNSAT	Periodic tiling	Total
20	962	770	488	2,220
21	38	56	11	105
22	53	85	10	148
23	76	102	4	182
24	57	57	22	136
25	107	95	8	210
26	70	69	15	154
27	90	65	10	165
28	62	52	8	122
Total	1,515	1,351	576	3,442

Table 7: Outcomes on the restricted collection. There are no unprocessed or undecided candidates in this collection. The periodic tilings use one (65 shapes), two (413) or four (98) copies per fundamental domain.

Of 1,862 shapes actually tested at $K = 3$, 511 are satisfiable and all 511 have explicit periodic tilings; 65 further shapes are eliminated by one-copy periodic tilings. One 20-cell shape tested at $K = 4$ is also satisfiable at $K = 5$ and has a four-copy periodic tiling with lattice basis $(40, 0), (18, 2)$. Its four-layer patch is consequently not evidence of a finite value four.

A separate finite audit checks the candidate identities, topology and all 576 periodic tilings. Fourteen UNSAT instances agree with a second SAT solver using the same encoding; two representative screening instances have drat-trim-accepted proofs. The remaining UNSAT outcomes are solver records without proof logs. All 513 saved satisfying patches (511 at $K = 3$, one at $K = 4$ and one at $K = 5$) pass geometric checking. These checks yield no finite-$H_{\mathrm{h}} = 4$ example. They do not establish that such a polyomino is impossible: the collection is neither an exhaustive size census nor a random sample.

11 Scope and remarks

What the certificates do not cover. The values are certified for the listed shapes only. That Kaplan's files contain every non-tiler of each size is taken from his enumeration and is not checked here. Polyominoes with 11–19 cells and $H_{\mathrm{h}} \leq 2$, polyhexes with 9–16 cells and $H_{\mathrm{h}} \leq 3$, polyhexes with 17 or more cells, and polyiamonds with 13–24 cells and $H_{\mathrm{h}} \leq 3$ are outside the selection. All statements concern copies on the fixed grid; a statement about arbitrary placements in the plane would need a further reduction.

Relation to other certification work. The Heesch challenge [13] provides, for individual shapes, a route through LRAT to the verified checker cake_lpr [20]; here every proof is checked by drat-trim, which is not formally verified, but the correspondence between geometry and CNF is a Lean theorem, and the correspondence between that theorem and the checked files is a recorded finite comparison. The empty hexagon verification [18] connects its Lean reduction to the external cake_lpr result through an explicit axiom; Szeider [19] streams LRAT certificates into Lean theorems, with native evaluation in the trusted base; and bv_decide [3] uses verified bit-blasting and LRAT checking inside Lean. Converting the 5,019 proofs to LRAT [6] and checking them with a verified checker, or inside Lean, would remove drat-trim

from the trusted base; replacing the file comparison by a Lean proof about the generating program would remove the transcription. Neither has been done.

Novelty. A literature search of published papers, preprints and public code up to 22 October 2026 found certificates for individual Heesch numbers [13] and Lean-verified reductions from combinatorial or geometric problems to SAT [5, 9, 18], but not the combination reported here: a fixed table range with witnesses, proofs and a second encoding for every value, and a Lean soundness theorem for a Heesch encoding on three grids connected to the checked files. Public corona enumeration reports [31] overlap the counting questions; Section 9 compares their values under explicit counting conventions. The restricted search is a record of the tested collection, not a new extremal construction. The search was not exhaustive, and we claim no priority beyond it.

Records. The following records accompany this article: the witnesses; the certificate records with the hashes of every CNF and proof; the Lean sources; the records of the file comparison; for the counts of Section 9, the cell coordinates of every enumerated inner part from both implementations, the per-inner-part outer completion counts, and the hashes and drat-trim results of the enumeration proofs; and for the search of Section 10, the candidate list with its SHA-256 hash, the classification of every candidate, the periodic tiling certificates and the saved satisfying patches.

AI-use statement

AI systems assisted with the computation of the values, witnesses and proofs, the second encoding and the geometric checkers, the Lean statement and proof, the independent reading of the Lean statement, the file comparison, maximal-corona enumeration and counting, candidate generation and screening, literature organization and manuscript drafting. The author is responsible for the claims and their stated scope; AI systems are not authors.

References

[1] B. Bašić, "A figure with Heesch number 6: pushing a two-decade-old boundary," *Math. Intelligencer* **43**(3), 50–53 (2021), doi:10.1007/s00283-020-10034-w.

[2] B. Bašić, "The Heesch number in $\mathbb{E}^3$ is unbounded," arXiv:2610.24281v1 (2026).

[3] H. Böving et al., "Interactive bitvector reasoning using verified bit-blasting," *Proc. ACM Program. Lang.* **9**(OOPSLA2), 3259–3285 (2025), doi:10.1145/3763167.

[4] C. R. Codel, *Verifying SAT Encodings in Lean*, M.S. thesis, Carnegie Mellon University (2022), `https://crcodel.com/research/verified_encodings_thesis.pdf`.

[5] C. R. Codel, J. Avigad and M. J. H. Heule, "Verified encodings for SAT solvers," in *Proc. FMCAD 2023*, 141–151, doi:10.34727/2023/isbn.978-3-85448-060-0_22.

[6] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt Jr., M. Kaufmann and P. Schneider-Kamp, "Efficient certified RAT verification," in *CADE-26*, LNCS **10395**, 220–236 (2017), doi:10.1007/978-3-319-63046-5_14.

[7] A. Fontaine, "An infinite number of plane figures with Heesch number two," *J. Combin. Theory Ser. A* **57**(1), 151–156 (1991), doi:10.1016/0097-3165(91)90013-7.

[8] H. Heesch, *Reguläres Parkettierungsproblem*, Westdeutscher Verlag, Köln/Opladen (1968), doi:10.1007/978-3-322-98547-7.

[9] W. H. Holliday, C. Norman, E. Pacuit and S. Zahedian, "Impossibility theorems involving weakenings of expansion consistency and resoluteness in voting," arXiv:2208.06907v3 (2023), Appendix C.

[10] C. S. Kaplan, "Heesch numbers of unmarked polyforms," *Contrib. Discrete Math.* **17**(2), 150–171 (2022), doi:10.55016/ojs/cdm.v17i2.72886; arXiv:2105.09438v1 (2021); data at `https://cs.uwaterloo.ca/~csk/heesch/`. Section numbers refer to arXiv:2105.09438v1.

[11] C. S. Kaplan, "The path to aperiodic monotiles," arXiv:2509.12216v1 (2025).

[12] isohedral/heesch-sat contributors, *heesch-sat*, software, `https://github.com/isohedral/heesch-sat`, commit `d12a527c04310fed51d522318126781e2f7ed56d`.

[13] Layr-Labs/Eigen and contributors, *Heesch challenge*, software and specifications, `https://github.com/Layr-Labs/heesch`, commit `947f05720e0842d80b5119b9b88546c890111d19`.

[14] A. Maiti, "Unboundedness of the Heesch number for hyperbolic convex monotiles," arXiv:2603.27827v3 (2026).

[15] C. Mann, "Heesch's tiling problem," *Amer. Math. Monthly* **111**(6), 509–517 (2004), doi:10.1080/00029890.2004.11920105.

[16] C. Mann and B. C. Thomas, "Heesch numbers of edge-marked polyforms," *Exp. Math.* **25**(3), 281–294 (2016), doi:10.1080/10586458.2015.1096867.

[17] C. Sinz, "Towards an optimal CNF encoding of Boolean cardinality constraints," in *CP 2005*, LNCS **3709**, 827–831, doi:10.1007/11564751_73.

[18] B. Subercaseaux, W. Nawrocki, J. Gallicchio, C. Codel, M. Carneiro and M. J. H. Heule, "Formal verification of the empty hexagon number," in *ITP 2024*, LIPIcs **309**, 35:1–35:19, doi:10.4230/LIPIcs.ITP.2024.35.

[19] S. Szeider, "Streaming LRAT certificates into Lean theorems," arXiv:2607.00815v2 (2026).

[20] Y. K. Tan, M. J. H. Heule and M. O. Myreen, "cake_lpr: verified propagation redundancy checking in CakeML," in *TACAS 2021*, LNCS **12652**, 223–241, doi:10.1007/978-3-030-72013-1_12.

[21] N. Wetzler, M. J. H. Heule and W. A. Hunt Jr., "DRAT-trim: efficient checking and trimming using expressive clausal proofs," in *SAT 2014*, LNCS **8561**, 422–429, doi:10.1007/978-3-319-09284-3_31.

[22] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *CADE 28*, LNCS **12699**, 625–635 (2021), doi:10.1007/978-3-030-79876-5_37.

[23] The mathlib Community, "The Lean mathematical library," in *CPP 2020*, 367–381, doi:10.1145/3372885.3373824; Lean 4 library: `https://github.com/leanprover-community/mathlib4`.

[24] Lean project contributors, *Comparator*, software, `https://github.com/leanprover/comparator`.

[25] M. Carneiro, "Lean4Lean: verifying a typechecker for Lean, in Lean," arXiv:2403.14064v3 (2025); software: `https://github.com/digama0/lean4lean`.

[26] Nanoda, ConLeche and ConRon contributors, external Lean checker software: `https://github.com/ammkrn/nanoda_lib`, `https://github.com/leanprover/con-leche`, `https://github.com/leanprover/con-ron`.

[27] A. Ignatiev, A. Morgado and J. Marques-Silva, "PySAT: a Python toolkit for prototyping with SAT oracles," in *SAT 2018*, LNCS **10929**, 428–437, doi:10.1007/978-3-319-94144-8_26.

[28] A. Biere and contributors, *CaDiCaL*, SAT solver software, `https://github.com/arminbiere/cadical`.

[29] G. Audemard and L. Simon, *Glucose*, SAT solver software, `https://www.cril.univ-artois.fr/software/glucose/`.

[30] A. Biere, *Lingeling*, SAT solver software, `https://github.com/arminbiere/lingeling`.

[31] Contributors to the Layr-Labs Heesch discussions, public enumeration reports, discussions #12, #32 and #94 (2026).